

Лабораторная работа №4

Постановка задачи:

Составить следующую программу, которая:

- ☐ принимая дескриптор, имя или полное имя модуля, возвращает другие два элемента в своих выходных параметрах (выполнить задание для своей программы и для любой известной библиотеки),
- ☐ будет выполнять последовательно по шагам следующее:
 1. используя функцию GetCurrentProcessId определит идентификатор текущего процесса.
 2. используя функцию GetCurrentProcess определит псевдодескриптор текущего процесса.
 3. используя функцию DuplicateHandle и значение псевдодескриптора определит дескриптора текущего процесса.
 4. используя функцию OpenProcess определит копию дескриптора текущего процесса
 5. закроет дескриптор, полученный функцией DuplicateHandle
 6. закроет дескриптор, полученный функцией OpenProcess
- ☐ выдает список перечисления всех процессов, потоков, модулей и их свойства в системе.

Код:

```
LR 4 (Глобальная область)
1  #include <iostream>
2  #include <windows.h>
3  #include <vector>
4  #include <tlhelp32.h>
5  #include <psapi.h>
6  #include <fstream>
7
8  using namespace std;
9
10
11  HANDLE duplicateProcessHandle(HANDLE hProcess) {
12      HANDLE hDuplicate = NULL;
13      if (DuplicateHandle(GetCurrentProcess(), hProcess, GetCurrentProcess(), &hDuplicate, 0, FALSE, DUPLICATE_SAME_ACCESS)) {
14          return hDuplicate;
15      }
16      else {
17          cerr << "Failed to duplicate handle!" << GetLastError() << endl;
18          return NULL;
19      }
20  }
21
22  HANDLE openProcess(DWORD processId) {
23      HANDLE hProcess = OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_VM_READ, FALSE, processId);
24      if (hProcess == NULL) {
25          cerr << "Failed to open process!" << GetLastError() << endl;
26      }
27      return hProcess;
28  }
29
30  void closeHandle(HANDLE hProcess) {
31      if (!CloseHandle(hProcess)) {
32          cerr << "Failed to close handle!" << GetLastError() << endl;
33      }
34  }
35
36  HANDLE createSnapshot() {
37      HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
38      if (hSnapshot == INVALID_HANDLE_VALUE) {
39          cerr << "Failed to create snapshot!" << GetLastError() << endl;
40      }
41      return hSnapshot;
42  }
43
44
45  void listProcesses() {
46      HANDLE hSnapshot = createSnapshot();
```

```

47     if (hSnapshot == INVALID_HANDLE_VALUE) {
48         return;
49     }
50
51     PROCESSENTRY32 pe32;
52     pe32.dwSize = sizeof(PROCESSENTRY32);
53
54     if (!Process32First(hSnapshot, &pe32)) {
55         cerr << "Failed to get first process!" << GetLastError() << endl;
56         closeHandle(hSnapshot);
57         return;
58     }
59
60     do {
61         wcout << L"Process Name: " << pe32.szExeFile << L" | Process ID: " << pe32.th32ProcessID << endl;
62     } while (Process32Next(hSnapshot, &pe32));
63
64     closeHandle(hSnapshot);
65
66 }
67
68 void getMemoryInfo(DWORD processId) {
69     HANDLE hProcess = openProcess(processId);
70     if (hProcess == NULL) {
71         return;
72     }
73
74     PROCESS_MEMORY_COUNTERS pmc;
75     if (GetProcessMemoryInfo(hProcess, &pmc, sizeof(pmc))) {
76         cout << "Process ID: " << processId << endl;
77         cout << "Working Set Size: " << pmc.WorkingSetSize << endl;
78         cout << "Pagefile Usage: " << pmc.PagefileUsage << endl;
79     }
80     else {
81         cerr << "Failed to get memory info for process with ID: " << processId << endl;
82     }
83
84     closeHandle(hProcess);
85 }
86
87
88 int main()
89 {
90
91     std::ofstream ofs;
92     ofs.open("status.txt", std::ofstream::out | std::ofstream::trunc);

```

```

92     ofs.open("status.txt", std::ofstream::out | std::ofstream::trunc);
93     ofs.close();
94
95     ofstream out("status.txt", std::ios::app);
96     if (out.is_open())
97     {
98         cout << "File output. Please wait.";
99
100         int n = 0;
101
102         MEMORYSTATUS memoryStatus = { 0 };
103         GlobalMemoryStatus(&memoryStatus);
104         out << "Physics memory available " << memoryStatus.dwAvailPhys / 1024 / 1024 << " MB " << '\n';
105         out << "Total physics memory " << memoryStatus.dwTotalPhys / 1024 / 1024 << " MB " << '\n';
106
107         out << "Virtual memory available " << memoryStatus.dwAvailVirtual / 1024 / 1024 << " MB " << '\n';
108         out << "Total virtual memory " << memoryStatus.dwTotalVirtual / 1024 / 1024 << " MB " << '\n';
109
110         out << "PageFile available " << memoryStatus.dwAvailPageFile << " out of " << memoryStatus.dwTotalPageFile << '\n';
111
112         SYSTEM_INFO siSysInfo;
113         GetSystemInfo(&siSysInfo);
114
115         LPVOID minAA = siSysInfo.lpMinimumApplicationAddress;
116
117         out << "Min Application Address " << minAA << '\n';
118
119         HANDLE CONST hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
120
121         out << "Current proces id: " << GetCurrentProcessId() << '\n';
122         out << "Artificial desc: " << GetCurrentProcess() << '\n';
123
124         HANDLE hProc = OpenProcess(PROCESS_ALL_ACCESS, FALSE, GetCurrentProcessId());
125
126         out << "Actual desc: " << GetCurrentProcess() << '\n';
127
128         vector<wchar_t> pathBuf;
129         DWORD copied = 0;
130         do {
131             pathBuf.resize(pathBuf.size() + MAX_PATH);
132             copied = GetModuleFileName(0, &pathBuf.at(0), pathBuf.size());
133         } while (copied >= pathBuf.size());
134
135         pathBuf.resize(copied);
136
137

```

```

138 wchar_t buffer[MAX_PATH];
139 GetModuleFileName(NULL, buffer, MAX_PATH);
140 out << "Module: " << buffer << '\n';
141
142 out << "Module handle: " << GetModuleHandle(buffer) << '\n';
143
144 string path(pathBuf.begin(), pathBuf.end());
145
146 out << "Path: " << path << '\n';
147
148 out << "\n-----MODULE INFO-----\n";
149
150 MODULEENTRY32 mEntry;
151 HANDLE CONST mSnap = CreateToolhelp32Snapshot(TH32CS_SNAPMODULE, 0);
152 Module32First(mSnap, &mEntry);
153
154 do {
155     out << "Module id: " << mEntry.th32ModuleID << '\n';
156     out << "Module process id: " << mEntry.th32ProcessID << '\n';
157     out << "Usages: " << mEntry.GblcntUsage << '\n';
158     out << "Module desc: " << mEntry.hModule << '\n';
159     out << "Module dw size: " << mEntry.dwSize << '\n';
160     out << "Module Base size: " << mEntry.modBaseSize << '\n';
161     out << "Module exe path: " << mEntry.szExePath << '\n';
162     out << "\n-----\n";
163 } while (Module32Next(mSnap, &mEntry));
164
165
166
167 PROCESSENTRY32 peProcessEntry;
168 peProcessEntry.dwSize = sizeof(PROCESSENTRY32);
169 Process32First(hSnapshot, &peProcessEntry);
170
171
172 DWORD processId = GetCurrentProcessId();
173
174 HANDLE hProcess = GetCurrentProcess();
175
176 HANDLE hDuplicate = duplicateProcessHandle(hProcess);
177 if (hDuplicate != NULL) {
178     cout << "Duplicate Process Handle: " << hDuplicate << endl;
179
180     HANDLE hOpenProcess = openProcess(processId);
181     if (hOpenProcess != NULL) {
182         cout << "Open Process Handle: " << hOpenProcess << endl;
183

```

```

184         closeHandle(hDuplicate);
185         cout << "Closed Duplicate Handle" << endl;
186
187         closeHandle(hOpenProcess);
188         cout << "Closed Open Process Handle" << endl;
189     }
190 }
191 listProcesses();
192
193 getMemoryInfo(processId);
194
195 return 0;
196
197 }
198
199
200 cout << "\nFile has been written" << endl;
201 cout << "Check out status.txt" << endl;
202 out.close();
203 int n;
204 cin >> n;
205 }

```

Результат:

```
Консоль отладки Microsoft V x + v
File output. Please wait.Duplicate Process Handle: 00000000000000BC
Open Process Handle: 00000000000000C0
Closed Duplicate Handle
Closed Open Process Handle
Process Name: [System Process] | Process ID: 0
Process Name: System | Process ID: 4
Process Name: Secure System | Process ID: 204
Process Name: Registry | Process ID: 276
Process Name: smss.exe | Process ID: 924
Process Name: csrss.exe | Process ID: 1168
Process Name: wininit.exe | Process ID: 1268
Process Name: services.exe | Process ID: 1340
Process Name: LsaIso.exe | Process ID: 1360
Process Name: lsass.exe | Process ID: 1376
Process Name: svchost.exe | Process ID: 1480
Process Name: fontdrvhost.exe | Process ID: 1512
Process Name: WUDFHost.exe | Process ID: 1564
Process Name: svchost.exe | Process ID: 1608
Process Name: svchost.exe | Process ID: 1664
Process Name: WUDFHost.exe | Process ID: 1732
Process Name: WUDFHost.exe | Process ID: 1788
Process Name: WUDFHost.exe | Process ID: 1848
Process Name: WUDFHost.exe | Process ID: 1900
Process Name: svchost.exe | Process ID: 1660
Process Name: svchost.exe | Process ID: 1836
Process Name: svchost.exe | Process ID: 2104
Process Name: svchost.exe | Process ID: 2160
Process Name: svchost.exe | Process ID: 2276
Process Name: svchost.exe | Process ID: 2288
Process Name: svchost.exe | Process ID: 2380
```

...

```
Process Name: browser.exe | Process ID: 10932
Process Name: VsDebugConsole.exe | Process ID: 22136
Process Name: conhost.exe | Process ID: 25940
Process Name: OpenConsole.exe | Process ID: 21068
Process Name: WindowsTerminal.exe | Process ID: 3216
Process Name: LR_4.exe | Process ID: 14688
Process Name: RuntimeBroker.exe | Process ID: 10292
Process ID: 14688
Working Set Size: 4038656
Pagefile Usage: 733184

D:\файлы (учеба)\программы\Visual Studio\проекты\LR_4\х64\Debug\LR_4.exe (процесс 14688) завершил работу с кодом 0.
Нажмите любую клавишу, чтобы закрыть это окно...
```

Файл:

```
status.txt x +
Файл Изменить Просмотр
Physics memory available 5157 MB
Total physics memory 16108 MB
Virtual memory available 134213579 MB
Total virtual memory 134217727 MB
PageFile available 3012923392 out of 18635546624
Min Application Address 000000000010000
Current proces id: 14688
Artificial desc: FFFFFFFFFFFFFFFF
Actual desc: FFFFFFFFFFFFFFFF
Module: 000000F90213E750
Module handle: 00007FF78DD80000
Path: D:\D09;K (CG510)\?@>3@<K\Visual Studio\?@>5:BK\LR_4\х64\Debug\LR_4.exe

-----MODULE INFO-----
Module id: 3435973836
Module process id: 3435973836
Usages: 3435973836
Module desc: CCCCCCCCCCCCCC
Module dw size: 3435973836
Module Base size: 3435973836
Module exe path: 000000F90213EBF0
-----
```

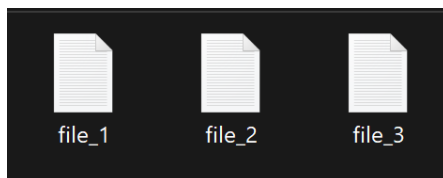
Лабораторная работа №5

Проиллюстрировать глобальность объекта ядра Mutex.

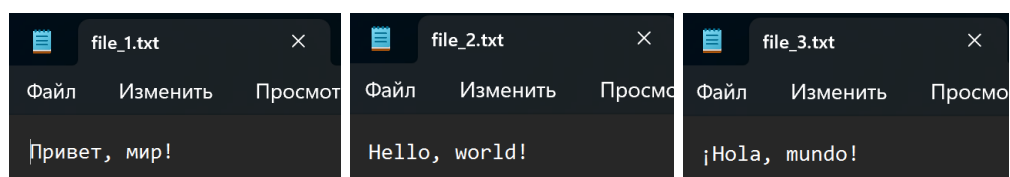
Код:

```
LR_5 (Глобальная область)
1  #include <iostream>
2  #include <fstream>
3  #include <string>
4  #include <vector>
5  #include <windows.h>
6
7  HANDLE mutex;
8
9  DWORD WINAPI CopyContents(LPVOID lpParam) {
10
11     std::cout << "\n" << "The copy function has started" << std::endl;
12     std::string* filePath = (std::string*)lpParam;
13     std::ifstream inFile(*filePath);
14     std::string content((std::istreambuf_iterator<char>(inFile), std::istreambuf_iterator<char>()));
15     inFile.close();
16
17
18     std::cout << "\n" << "Start of synchronization" << std::endl;
19     WaitForSingleObject(mutex, INFINITE);
20     std::ofstream outFile("conclusion.txt", std::ios::app);
21     outFile << content << "\n";
22     outFile.close();
23     ReleaseMutex(mutex);
24     std::cout << "End of synchronization" << std::endl;
25
26     return 0;
27 }
28
29 int main() {
30
31     std::vector<std::string> fileNames = { "file_1.txt", "file_2.txt", "file_3.txt" };
32     std::vector<HANDLE> threads(fileNames.size());
33     HANDLE mutex = CreateMutex(NULL, FALSE, NULL);
34     std::cout << "A mutex has been created for synchronization" << std::endl;
35
36
37     for (size_t i = 0; i < fileNames.size(); ++i) {
38         std::cout << "Stream " << i << " has been created" << "\n" << std::endl;
39         threads[i] = CreateThread(NULL, 0, CopyContents, &fileNames[i], 0, NULL);
40     }
41
42
43     WaitForMultipleObjects(threads.size(), &threads[0], TRUE, INFINITE);
44     std::cout << "Termination of all threads" << std::endl;
45
46
47     for (HANDLE thread : threads) {
48         CloseHandle(thread);
49     }
50     std::cout << "All stream descriptors are closed" << std::endl;
51
52     CloseHandle(mutex);
53     std::cout << "The mutex is closed" << std::endl;
54     return 0;
55 }
```

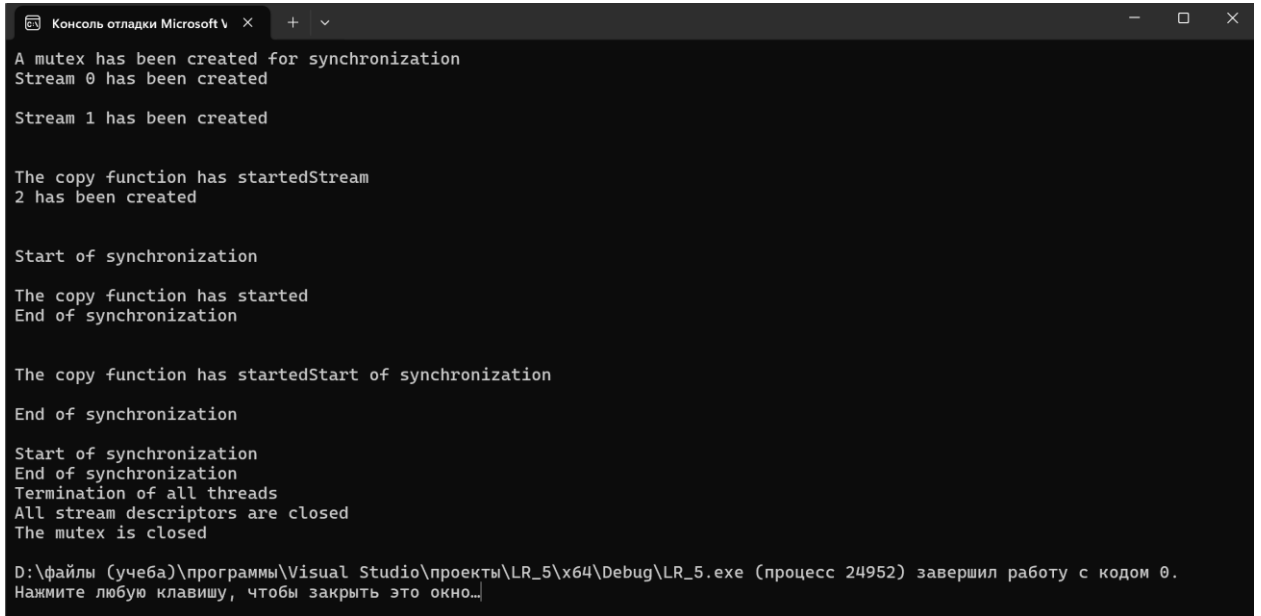
Предварительно созданные файлы .txt



Предварительное заполнение файлов



Результат:



```
Консоль отладки Microsoft V  X + -
A mutex has been created for synchronization
Stream 0 has been created

Stream 1 has been created

The copy function has startedStream
2 has been created

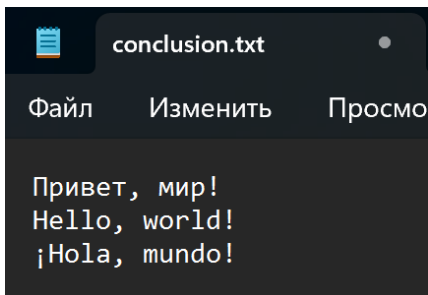
Start of synchronization
The copy function has started
End of synchronization

The copy function has startedStart of synchronization
End of synchronization

Start of synchronization
End of synchronization
Termination of all threads
All stream descriptors are closed
The mutex is closed

D:\файлы (учеба)\программы\Visual Studio\проекты\LR_5\x64\Debug\LR_5.exe (процесс 24952) завершил работу с кодом 0.
Нажмите любую клавишу, чтобы закрыть это окно...
```

Результат внутри созданного в программе файла Conclusion.txt



```
conclusion.txt
Файл  Изменить  Просмо
Привет, мир!
Hello, world!
¡Hola, mundo!
```